

Rapport technique - Satisfaction Client

1. Objectif du projet

Le projet Satisfaction Client transforme des avis clients non structurés en indicateurs exploitables pour une entreprise. Il permet de collecter ou importer des avis, de les analyser avec un modèle de sentiment propriétaire, de restituer les résultats dans une application web et d'alimenter une boucle d'amélioration continue via des corrections humaines.

La solution répond à une problématique de satisfaction client en aval de la supply chain : comprendre rapidement les irritants liés à la livraison, au service client, au prix, au remboursement, à la qualité produit ou à la conformité des commandes.

2. Architecture générale

Le projet est orchestré avec Docker Compose et se compose de plusieurs services complémentaires.

Service	Technologie	Rôle
frontend	React, Vite, TypeScript	Interface produit pour lancer, comparer et corriger les analyses.
api	FastAPI	API produit, endpoints métier protégés par JWT et rôles.
celery_worker	Celery	Exécution asynchrone des analyses et des reentrainements.
redis	Redis	Broker et backend de résultats Celery.
postgres_db	PostgreSQL	Stockage des entreprises, runs, avis, prédictions et corrections.
mlflow	MLflow	Suivi des expérimentations et registre de modèle.
worker	Python	Scripts historiques de scraping, ETL et entraînement.
dashboard	Streamlit	Ancienne interface d'exploration, conservée comme support historique.

Flux principal :

```
Utilisateur
-> React
-> FastAPI
-> PostgreSQL
-> Celery / Redis
-> Scraping Trustpilot ou import CSV
-> Prediction sentiment via modèle MLflow
-> PostgreSQL
-> Rapport React, export CSV/PDF, corrections humaines
```

3. Sources de données

Deux modes d'acquisition sont disponibles.

3.1 Trustpilot

L'utilisateur saisit une entreprise ou une URL Trustpilot. L'API crée un run d'analyse et Celery exécute le scraping avec Playwright. Le scraping peut cibler plusieurs pages par note, ce qui permet d'équilibrer les avis entre 1 et 5 étoiles.

Données collectées :

- auteur ;
- note ;
- date brute ;
- verbatim ;
- indicateur de réponse entreprise ;
- entreprise associée.

Une collecte privée distincte du pipeline produit a ensuite parcouru les 16 vues linguistiques accessibles de www.vapoter.fr sur Trustpilot jusqu'à leur fin naturelle. Elle a sauvegardé 476 pages et, après réconciliation hors ligne, 11

281 identifiants uniques (9 261 avis principaux et 2 020 empiles), dont 9 473 avec reponse d'entreprise. La preuve assainie publie les agregats et empreintes sans texte d'avis. Le corpus n'a pas ete importe en base ni utilise pour l'entrainement ML.

Le controle hors ligne des notes donne 289 avis a 1 etoile, 115 a 2, 239 a 3, 1 030 a 4 et 9 608 a 5. La definition operationnelle retenue pour les avis negatifs est **1 ou 2 etoiles** : 404 avis, dont 360 avec reponse d'entreprise et 44 sans reponse, soit 89,11 % de reponses visibles. Ce resultat de collecte privée n'est pas un KPI produit calcule par le dashboard.

Une observation ponctuelle du profil Trustpilot Vapoter, le 30 septembre 2026 a 16:27 (Europe/Paris), a releve le nom commercial Vapoter, la categorie « Magasin de cigarettes electroniques », un TrustScore de **4,8 / 5** et **11 281 avis affiches**. La repartition affichee etait : 1 etoile 3 % (289 avis), 2 etoiles 1 % (115), 3 etoiles 2 % (239), 4 etoiles 9 % (1 030), 5 etoiles 85 % (9 608) ; les comptes proviennent des infobulles. Cette concordance chiffreée avec le corpus ne prouve pas un instantane identique d'identifiants. Le nom juridique complet reste non demontre.

3.2 CSV utilisateur

L'utilisateur peut importer un fichier CSV d'avis. Ce mode permet d'analyser des exports clients ou des avis provenant d'autres plateformes sans dependre de Trustpilot.

Le controle avant import verifie :

- le nombre total d'avis ;
- le nombre de lignes exploitables ;
- le nombre de lignes ignorees ;
- les colonnes detectees ;
- un aperçu des premiers verbatims.

Les colonnes minimales attendues sont :

- texte de l'avis ;
- note, si disponible.

Les colonnes optionnelles sont :

- auteur ;
- date ;
- reponse entreprise.

4. Modele de donnees

L'application produit utilise un schema PostgreSQL dedie de **16 tables**, versionne par huit migrations Alembic jusqu'a 20260921_0008. Le tableau ci-dessous ne presente que les tables centrales de l'analyse ; le catalogue complet, ses cardinalites, contraintes et diagrammes est egalement livre en PDF.

Table	Role
companies	Entreprises analysees et source associee.
analysis_runs	Historique des analyses, statut, duree, source et fichiers produits.
analysis_run_events	Journal d'execution lisible par l'utilisateur.
reviews	Avis individuels rattaches a un run.
sentiment_predictions	Prediction de sentiment et score du modele.
review_topics	Irritants detectes sur chaque avis.
review_feedback	Corrections humaines pour ameliorer le modele.
model_training_runs	Historique des reentrainements lances depuis l'interface.

Ce modele separe les donnees brutes, les predictions, les corrections et les metadonnees d'execution. Il permet de conserver un historique auditable, de comparer plusieurs entreprises et de reutiliser les corrections pour le reentrainement.

Le schema produit est versionne avec Alembic. Une base existante non versionnee est validee puis rattachee a la baseline sans recreation des tables. Les tables historiques `dim_companies` et `fact_reviews` restent definies par `init_db.sql`

pour les anciens scripts. Elles ne sont pas les tables du produit. Le ON CONFLICT de l'ancien app/etl.py vise une unicite absente du SQL historique ; ce flux ne doit pas etre presente comme le chemin produit valide.

5. Pipeline d'analyse

1. L'utilisateur cree une analyse depuis le frontend.
2. FastAPI valide la demande et cree un analysis_run.
3. Si une analyse identique est deja active, l'API renvoie un conflit pour eviter les doublons.
4. Le run est envoye dans Celery.
5. Le worker collecte les avis ou relit le CSV importe.
6. Les avis sont nettoyes et normalises.
7. Le modele de sentiment charge depuis MLflow predit un label et un score.
8. Des irritants metier sont detectes par analyse lexicale.
9. Les resultats sont stockes dans PostgreSQL.
10. Le frontend affiche le rapport, les avis, les exports et le journal d'execution.

Le diagramme ETL detaille distingue la collecte Trustpilot, l'import CSV, la normalisation, les tables produit PostgreSQL, l'inference MLflow et la restitution API/frontend. Le lot historique de 1 200 avis a une provenance d'acquisition incomplete ; l'ancienne tentative Showroomprive HTTP 403 n'a extrait aucun avis. Ces deux JSON ne sont pas la preuve finale. La collecte privree Vapoter a parcouru les 16 vues accessibles jusqu'a leur fin naturelle : 476 pages, 11 281 IDs uniques et 0 collision inter-corpus apres reconciliation. Les compteurs UI ont evolue durant la collecte ; ce resultat n'est pas un instantane parfaitement simultane.

Les statuts principaux d'un run sont :

- pending ;
- running ;
- completed ;
- failed ;
- empty.

6. Modele de sentiment

Le modele est un classifieur supervise scikit-learn entraine sur des avis annotes.

Caracteristiques :

- vectorisation TF-IDF des verbatims ;
- prise en compte reduite de la note client ;
- classification en trois classes : Negatif, Neutre, Positif ;
- evaluation par accuracy, precision, recall et F1-score ;
- serialisation locale ;
- publication dans MLflow avec alias de production.

La note client est volontairement ponderee afin que le texte garde une influence centrale. Ce choix repond au besoin metier : detecter les cas ou la note et le verbatim racontent des choses differentes.

7. Boucle d'amelioration humaine

L'application permet de corriger manuellement le sentiment d'un avis. Les corrections sont stockees dans review_feedback, exportables en CSV et integrees au prochain reentrainement.

Le reentrainement :

- combine le corpus historique annote et les corrections humaines ;
- pondere davantage les corrections recentes ;
- cree un snapshot auditable du dataset d'entrainement ;

- évalue le modèle sur un split stratifié ;
- peut enregistrer une nouvelle version candidate dans MLflow ;
- ne met à jour l'alias de production et ne resynchronise les prédictions qu'après promotion explicite.

Cette boucle transforme l'application en outil d'amélioration continue plutôt qu'en simple dashboard statique.

8. API FastAPI

L'API expose les principaux endpoints suivants.

Endpoint	Role
GET /health	Vérification de disponibilité.
POST /analysis-runs	Création d'une analyse Trustpilot.
POST /analysis-runs/preview-csv	Contrôle avant import CSV.
POST /analysis-runs/import-csv	Import et analyse d'un CSV.
GET /analysis-runs	Historique des analyses.
GET /analysis-runs/{run_id}	Détail d'un run.
POST /analysis-runs/{run_id}/execute	Relance d'un run.
GET /analysis-runs/{run_id}/events	Journal d'exécution.
GET /analysis-runs/{run_id}/summary	Rapport synthétique.
GET /analysis-runs/{run_id}/reviews	Avis pages et filtrables.
POST /analysis-runs/{run_id}/reviews/{review_id}/feedback	Correction humaine.
GET /analysis-runs/feedback/quality	Synthèse qualité IA.
GET /analysis-runs/compare	Benchmark multi-entreprises.
GET /model-training/overview	État du modèle de production.
POST /model-training/runs	Lancement d'un reentrainement.

Les endpoints métier sont protégés par des jetons JWT et des rôles, avec isolation par organisation. L'endpoint /health reste public afin de faciliter les sondes de disponibilité.

9. Interface React

Le frontend React/Vite/TypeScript est l'interface principale du produit. Il permet :

- de lancer une analyse Trustpilot ;
- d'importer un CSV avec contrôle avant import ;
- de suivre l'historique des runs ;
- de consulter le journal d'exécution ;
- de visualiser les KPIs d'un rapport entreprise ;
- de comparer plusieurs runs dans un benchmark ;
- de corriger les labels d'avis ;
- de piloter la qualité IA ;
- de déclencher un reentrainement ;
- d'exporter les avis, les corrections et les rapports.

L'interface vise un usage par des profils métier : responsable service client, responsable supply chain, analyste data ou direction.

10. Restitution métier

Un rapport entreprise contient :

- volume d'avis analyses ;
- note moyenne ;
- confiance IA moyenne ;
- repartition des sentiments ;
- repartition par note ;
- irritants principaux ;
- avis critiques ;
- incoherences note / texte ;
- score sante ;
- priorites recommandees ;
- points de vigilance.

Le benchmark multi-entreprises permet de comparer deux a quatre runs et de detecter :

- l'entreprise la plus a risque ;
- les irritants communs ;
- les irritants propres ;
- les differences de repartition negative ;
- les signaux metier prioritaires.

11. Dockerisation et reproductibilite

Le projet se lance avec Docker Compose. Les services applicatifs, la base de donnees, Redis, MLflow et le frontend sont definis dans `docker-compose.yml`.

Commandes principales :

```
docker-compose up -d --build api celery_worker frontend
docker-compose up -d postgres_db mlflow redis
docker-compose logs -f api
docker-compose logs -f celery_worker
```

Cette organisation rend le projet reproductible sur une autre machine equipee de Docker.

12. CI/CD et qualite

La CI GitHub Actions execute les tests backend, les tests et le build frontend, puis construit l'image Docker. Un job e2e-smoke manuel valide un demarrage Compose isole et un import CSV de 15 avis. Le workflow de deploiement manuel vise une VM de test existante et verifie API et frontend. Les runs exacts et leurs limites sont dans les preuves DevOps.

Les validations disponibles incluent :

- build du frontend ;
- compilation Python de l'API ;
- verification `git diff --check` ;
- migrations Alembic sur bases PostgreSQL temporaires ;
- test de l'endpoint `/health` ;
- execution d'analyses Trustpilot ;
- import CSV ;
- correction humaine ;
- reentrainement ;
- export CSV et PDF.

Le script KPI extrait les temps de deploiement des runs GitHub Actions. La supervision de test archive un JSON et un Step Summary. Le cron final de Monitor est `17 * * * *` ; le workflow diagnostic heartbeat a ete retire. Le heartbeat planifie 35765192366 a reussi et l'evenement Monitor planifie 35764642522 a ete cree, mais son job a ete skipped par `TEST_MONITOR_SCHEDULE_ENABLED=false`. La sonde manuelle reelle 35726582579 a reussi ; aucune sonde VM

planifiée active n'est prouvée. Le restart conditionnel unique de l'API ou du frontend est testé par le code, sans restart réel après incident démontré sur la VM.

12.1 Interruption mesurée pendant un déploiement

Le 30 septembre 2026, deux sondes HTTP indépendantes ont interrogé chaque seconde environ l'API /health et le frontend / sur la VM de test. Elles ont démarré avant le workflow de déploiement 36788014928 et continué après ses healthchecks. Le SHA servi est passé de 72517acbf156efab454e6496670b67526b109c3f a a890033e4b178bb1a56d1de28618662372812df4.

L'API a connu 20 échecs sur 313 sondes : de la première requête non saine au premier succès suivi de trois succès consécutifs, l'interruption observée est de **21,177 s**. Le frontend a répondu HTTP 200 sur **320/320 sondes** : aucune interruption observée à la résolution nominale d'environ une seconde. Le build précède le remplacement des conteneurs par Compose, sans docker compose down global ; les healthchecks ont validé le retour en service. Le rollback applicatif était disponible, mais n'a pas été utilisé.

Le mécanisme actuel limite l'interruption sans l'éliminer. Cette mesure sur une architecture mono-instance ne prouve ni blue/green, ni haute disponibilité, ni garantie de zéro interruption. La panne initiale de la VM, restaurée avant le test par démarrage des conteneurs existants, est exclue de ces métriques. Protocole, agrégats et limites figurent dans la preuve de downtime assainie.

13. Sécurité et cadre réglementaire

Mesures déjà présentes :

- authentification JWT locale, rôles et isolation par organisation ;
- endpoint de santé public limité ;
- variables d'environnement et secrets GitHub pour la configuration de test ;
- séparation des services Docker ;
- export explicite des données.

Points à renforcer pour une production réelle :

- remplacer les secrets de développement par un gestionnaire de secrets ;
- renforcer l'authentification et la gestion des secrets pour une production réelle ;
- documenter la durée de conservation des avis ;
- anonymiser les auteurs si le contexte d'usage l'exige ;
- formaliser les contraintes RGPD.

14. Limites connues

- Le scraping Trustpilot dépend de la structure HTML du site.
- L'ancienne tentative Showroomprive a reçu HTTP 403 ; la collecte privée Vapoter démontre ensuite plus de 10 000 IDs sur les vues accessibles. Les métadonnées de profil sont une observation ponctuelle ; le nom juridique complet et les droits de republication des textes restent à documenter.
- Les avis ironiques ou très courts restent difficiles à classer.
- La classe Neutre est plus difficile à apprendre.
- Les corrections humaines doivent rester cohérentes pour ne pas dégrader le modèle.
- Le monitoring GitHub Actions concerne la VM de test, pas un SLA de production ; des événements schedulés sont prouvés, mais pas encore une sonde planifiée active de la VM.
- Le déploiement automatisé cible une VM scolaire existante, sans provisioning cloud.
- La mesure de déploiement porte sur un seul run de test ; l'API a subi 21,177 s d'indisponibilité observée, sans garantie de continuité de tous les parcours métier.
- Le rollback DB/migrations n'a pas été démontré.
- Le rollback applicatif a été testé entre deux révisions Git dont la seconde est un commit vide ; le retour de fonctionnalités différentes n'est pas prouvé.

- La v53 correspond au registre MLflow conserve ; un cold start sur registre vierge cree une nouvelle v1 et non une copie de cet historique.

15. Perspectives

Les evolutions les plus pertinentes sont :

1. documenter les droits de reutilisation et, si necessaire, le nom juridique de l'entreprise, puis etudier la representativite temporelle du corpus prive ;
2. enrichir le corpus avec plus d'entreprises et de secteurs ;
3. ajouter une classification thematique plus robuste ;
4. brancher d'autres sources d'avis via API ou CSV ;
5. verifier une sonde planifiee active avec le cron horaire deja retabli ;
6. preparer un eventuel provisioning cloud distinct de la VM scolaire ;
7. formaliser RGPD, retention et securite de production ;
8. ajuster les restart policies pour le retour automatique des services apres reboot, puis envisager plusieurs instances avec bascule de trafic si une disponibilite plus forte devient necessaire.

16. Conclusion technique

Le projet couvre les principales etapes attendues : collecte, organisation, traitement, machine learning, restitution, API, dockerisation et boucle d'amelioration. Il a evolue d'un pipeline de scraping vers une application produit exploitable par une entreprise pour analyser sa satisfaction client et prioriser ses actions.